

API hooking - code custom

samedi 14 mars 2026 00:37

Le but de l'api hooking est de "dévier"

La fonction initiale en faisant un jmp vers une autre fonction. C'est comme le code cave sauf qu'on utilise pas les codes caves.

C'est une technique dite compliquée mais en fait c'est simple et ici on va faire de l'API hooking en codant nous même sans utiliser des bibliothèques du style detours ou Minhook.

Bref. ON a plusieurs manières de faire un API Hooking. En premier lieu on doit d'abord réécrire l'instruction avec un jmp dit "trampoline" vers l'adresse de notre fonction malicieuse.

ON va le faire en assembleur :

#1 Trampoline Shellcode :

d'abord on met l'adresse de notre fonction malicieuse dans un registre avec une instruction mov :

Si on est sur une architecture de 64bits on utilise le registre r10 par exemple
Si on est sur une architecture à 32bits, on utilise le registre eax par exemple.

Puis on utilise l'instruction jmp vers cette adresse.

CE qui nous donne :

```
mov r10, pAddress
jmp r10
```

Avec pAddress qui contient l'adresse de la fonction .

Comme c'est un shellcode, on doit le convertir en opcodes.

Ce qui nous donne

64-bit :

```
0x49, 0xBA, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, // mov r10, pAddress
0x41, 0xFF, 0xE2 // jmp r10
```

32-bit :

```
0xB8, 0x00, 0x00, 0x00, 0x00, // mov eax, pAddress
0xFF, 0xE0 // jmp eax
```

Ici le pAddress est composé de NULL (0x00) car il va se remplir pendant le runtime du programme.

On utilise ici uint8_t, uint32 et uint64_t et pas int parce qu'on veut stocker des raw bytes. Et le int ne permet pas de gérer ça.

64 bit :

```
UInt8_t Trampoline [] = {
0x49, 0xBA, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, // mov r10, pAddress
0x41, 0xFF, 0xE2 // jmp r10 };
```

UInt64_t Patch = (uint64_t)pAddress; //on save l'adresse de notre malicieux shellcode pour le patcher plus tard

Entre temps on fait un GetProcAddress pour récupérer l'adresse.

Ensuite on copie l'adresse de la fonction dans la deuxième case de notre tableau trampoline.

Memcpy(&Trampoline[2], &patch, sizeof(patch)); //on copie ici l'adresse de notre malicieux shellcode dans la deuxième adresse (offset) du tableau trampoline.

32 bit :

```
UInt8_t Trampoline [] = {
0xB8, 0x00, 0x00, 0x00, 0x00, // mov eax, pAddress
0xFF, 0xE0 // jmp eax
};
```

UInt32_t Patch = (uint64_t)pAddress; //on save l'adresse de notre malicieux shellcode dans memcpy(&trampoline[1], &patch, sizeof(patch));

Pour rappel, pAddress est l'adresse de la fonction ou l'on va jumper. Memcpy permet de copier l'adresse de cette fonction dans notre tableau trampoline. CE qui va faire qu'on va jumper à cette adresse ducoup.

Avant d'utiliser le trampoline il faut rendre la zone écrivable et exécutable pour qu'on puisse déjà remplacer la fonction et ensuite l'exécuter.

Pour ça on va utiliser la fonction VirtualProtect pour changer la protection :

```
If(!VirtualProtect(pfunctionToHook, sizeof(trampoline), PAGE_EXECUTE_READWRITE, &oldprotection)){
Return false; }
```

La variable oldprotection est hyper importante car elle va nous permettre de remettre l'ancienne protection après...

Ensuite on va copier le contenu de notre trampoline dans la variable fonctionToHook qui contient l'adresse de notre fonction qu'on va hooker.

Memcpy(&pfunctionToHook, &trampoline, sizeof(trampoline));

OK maintenant qu'on a dévié (hooker) notre fonction, on va devoir la redévier vers la vraie fonction pour reprendre la suite du processus (comme un code cave avec l'adresse de retour)

Donc pour ça on va récupérer les octets d'origines qui auront avant de hooker été sauvegardé dans un buffer puis on va remettre les anciennes protections. Ce buffer doit faire la même taille que le trampoline comme ça n'overwrite pas le reste quand on fera le unhooking. Ce qui nous donne :

```
Memcpy(pfunctionToHook, pOriginalBytes, sizeof(pOriginalBytes));
```

Puis on remet l'ancienne protection :

```
If(!VirtualProtect(pfunctionToHook, sizeof(trampoline), oldprotection &oldprotection)){
Return false; }
```

Bon pour nous faciliter la tâche on peut utiliser une structure pour initialiser plus simplement :

```
#ifdef _M_X64
#define TRAMPOLINE_SIZE 13
#endif
```

```
#ifdef _M_X32
#define TRAMPOLINE_SIZE 7
#endif
```

```
typedef struct st_hook {
```

```
PVOID pfunctionToHook;
PVOID pfunctionToRun;
BYTE poriginalBytes[TRAMPOLINE_SIZE];
DWORD OldProtection;
```

```
}hook, *phook;
```

EN totalité on a :

```
#include <stdio.h>
#include <windows.h>
#include <stdlib.h>
#pragma comment(lib, "User32.lib")
typedef unsigned long long uint64_t;
typedef unsigned int      uint32_t;
typedef unsigned char      uint8_t;
#ifdef _M_X64
#define TRAMPOLINE_SIZE 13
#endif
#ifdef _M_X86
#define TRAMPOLINE_SIZE 7
#endif

typedef struct st_hook{
PVOID pfunctionToHook;
PVOID pMaliciousfunctionToRun;
BYTE OriginalBytes[TRAMPOLINE_SIZE];
DWORD OldProtection;
}hook, *phook;

//notre fonction malicieuse (remplacer par un shellcode)
INT WINAPI MyMessageBoxA(HWND hWnd, LPCSTR lpText, LPCSTR lpCaption, UINT uType) {
printf("[+] Original Parameters : \n");
printf("\t - lpText : %s\n", lpText);
printf("\t - lpCaption : %s\n", lpCaption);
return MessageBoxW(hWnd, L"je suis mechante", L"Hooked MsgBox", uType);
}

int main(int argc, char* argv[]){
hook Hook = {0};
Hook.pMaliciousfunctionToRun = (PVOID)&MyMessageBoxA;
Hook.pfunctionToHook = (PVOID)&MessageBoxA;

//on va créer le trampoline jmp
#ifdef _M_X64
uint8_t trampoline [] = {0x49, 0xBA, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00};
// mov r10, pFunctionToRun
0x41, 0xFF, 0xE2 };
//patch récupère l'adresse de notre fonction malicieuse
uint64_t patch = (uint64_t)Hook.pMaliciousfunctionToRun;

//on décode la fonction avant de la copier dans le trampoline :

//on met dans la deuxième adresse de trampoline notre fonction malicieuse grâce à
patch
memcpy(&trampoline[2], &patch, sizeof(patch));
#endif
#ifdef _M_X86
uint8_t trampoline [] = {0xB8, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00};
// mov eax, pFunctionToRun
0xFF, 0xE0}; // jmp eax
uint32_t patch = (uint32_t)Hook.pMaliciousfunctionToRun;
memcpy(&trampoline[1], &patch, sizeof(patch)); //on met l'adresse de notre shellcode
dans le trampoline
#endif
//on sauvegarde la fonction victime dans la variable original bytes
memcpy(Hook.OriginalBytes, Hook.pfunctionToHook, TRAMPOLINE_SIZE); // save

//on va changer la protection de la zone mémoire de la fonction à hooker :
```

```
//la fonction victime est de base dans la zone mémoire avec une certaine protection
//on va la changer pour nous permettre d'écraser cette fonction avec notre mauvaise fonction
if(!VirtualProtect(Hook.pfunctionToHook, TRAMPOLINE_SIZE, PAGE_EXECUTE_READWRITE,
&Hook.OldProtection)){
    return 1;
}
//comme on a changé la protection de la zone mémoire on se trouvait la fonction victime on
peut l'écraser avec notre trampoline
//maintenant on va mettre le trampoline à l'adresse de la vraie fonction à dévier
memcpy(Hook.pfunctionToHook, trampoline, TRAMPOLINE_SIZE); // hook

//comme notre fonction victime à maintenant un jmp vers notre fonction malicieuse
//si on appelle notre fonction victime, le rip va suivre le jmp vers notre fonction malicieuse
MessageBoxA(NULL, "What Do You Think About Malware Development ?", "Original MsgBox", MB_OK |
MB_ICONQUESTION);
//normalement on devrait tomber sur notre fonction malicieuse
//ensuite on va unhooker notre fonction
printf("on va unhooker la fonction :");
getchar();

memcpy(Hook.pfunctionToHook, Hook.OriginalBytes, TRAMPOLINE_SIZE); // unhook
//on va nettoyer le buffer
memset(Hook.OriginalBytes, 0, TRAMPOLINE_SIZE);
//on remet la protection de base de la fonction victime
if(!VirtualProtect(Hook.pfunctionToHook, TRAMPOLINE_SIZE, Hook.OldProtection,
&Hook.OldProtection)){
    return 1;
}
//et on rappelle notre fonction victime et normalement elle s'exécutera normalement (notre
fonction victime ici est messagebox)
MessageBoxA(NULL, "Normal MsgBox Again", "Original MsgBox", MB_OK | MB_ICONINFORMATION);

return 0;
}
```


